Computer Science Project File

FinVault - Bank Management System

2024-2025

ASIAN INTERNATIONAL PRIVATE SCHOOL

Ruwais, Abu Dhabi UAE

NAME: DAKSH KOTHARI
GRADE: XII
REGISTER NUMBER:
GUIDED BY: Ms. RESHMA PREMARAJAN

ASIAN INTERNATIONAL PRIVATE SCHOOL-RUWAIS

CERTIFICATE

This is to certify that ~~Miss~~/Master Daksh Kothari of Grade **12** has completed the project work in Fin-Vault Bank Management System (Python and SQL connectivity) prescribed by the Central Board of Secondary Education, New Delhi during the academic year 2024-25.

Teacher –in-charge : ……. **Date: …….**

External Examiner: …….

Principal: **School Seal:**

Table of Contents

SL NO:	TOPICS
1.	Acknowledgement
2.	Objective
3.	Abstract
4.	Packages Used
5.	Files Generated
6.	Methods Used
7.	Source Code
8.	Output Screens
9.	Limitations
10.	Requirements
11.	Bibliography

Acknowledgement

In the accomplishment of this project, many people have bestowed upon their blessings and their heart pledged support.

Primarily I thank God Almighty for being able to complete this project with success. Then I would like to thank the management, my Principal Mr. Anzar Abdul Salam and my Computer Science teacher Ms. Reshma Premarajan whose valuable guidance and support has helped me bring out this project. Their suggestions and instructions have served me towards the completion of this project. I would also like to thank my parents and friends for encouraging me during the various phases of this project. Finally I would like to thank CBSE for giving me this opportunity to undertake this project.

Objective

To develop a Python and SQL-based Bank Management System that efficiently manages customer accounts, transactions, and financial records with user-friendly functionality.

Abstract

The **"Bank Management System"** is a comprehensive application built using Python and SQL to automate banking operations, manage user accounts, and provide financial tracking tools with a user-friendly interface.

Key Components:

1. **User Accounts**:
   - **Customer Accounts**:
     Customers can create accounts, perform banking operations, and track finances.
   - **Admin Accounts**:
     Administrators have special access to manage the system and oversee customer data.

2. **Core Banking Operations**:
   - **Account Management**:
     Users can open accounts, log in, and access their account details.
   - **Transactions**:
     Customers can deposit and withdraw money securely.

3. **Expense Management**:
   - Users can log personal expenses for tracking and budgeting purposes.
   - A dedicated feature to track and categorize expenses.

4. **Bank Statements**:
   - Generate detailed statements showing transaction history for better financial understanding.

5. **Security Features**:
   - Includes a login system to ensure user data privacy and secure operations for both customers and administrators.

6. **Interface Design**:
   - Stylish and interactive menus with custom ASCII art to enhance usability.
   - Well-organized options for smooth navigation.

Customer's Perspective:

- Create accounts, log in, and manage finances.
- Deposit and withdraw balances as needed.
- Log and track expenses.
- Access bank statements and account details.

Admin's Perspective:

- Log in securely to manage customer accounts.
- Oversee and update system information.

This system delivers a reliable and user-centric solution for managing banking operations and personal finances, enhancing the banking experience for both customers and administrators.

Packages Used

1. mysql.connector (Database Interaction):
- Connecting Python to MySQL and committing SQL queries from python IDE.

2. random (Random Number Generation):
- Used to generate unique and random Bank Account IDs

3. csv (CSV File Handling):
- Storing Data for Account Username and Password.

4. datetime and date (Date and Time Operations):
- Helps manage and compare dates for transactions, track expenses, and generate time-based reports.

5. tabulate (Tabular Data Formatting):
- Used to display account details, transaction histories, and financial data in a neatly formatted table for better readability.

6. Pillow (PIL) (Image Processing):
- To show a welcome image.

7. matplotlib.pyplot (Data Visualization):
- To show data analytics for the transactions done.

8. time (Time Management):
- To get the time of the transaction being recorded.

9. prettytable (Tabular Data Representation):
- Used to present tabular data of account details and transactions done

Files Generated

Files:

1. **main.py:** Main python file having the source code of the project.

2. **help_file.txt**: Generate Help Information.

3. **image.png**: To showcase a welcome image right after executing the program.

4. **data.csv**: For storing data related to account username and password.

SQL Tables:

1. **User - Specific Tables**:
   To show transaction details of depositing, Withdrawing and Logging transactions.

2. **Users Table:**
   To store user details which are Account number, balance, Date of Joining and Username.

Methods Used

1. **display_image(image_path):**
   Displays welcome image using pillow module.
2. **generate_account(cursor):**
   Generates a unique 4-digit account number that is verified against the database to avoid duplicates.
3. **fin_vault():**
   Prints the application title and branding using ASCII art.
4. **welcome():**
   Displays the main menu and prompts the user to select actions
5. **signup():**
   Allows users to create a new account by providing a username and password.
6. **login():**
   Authenticates existing users by checking their username and password against stored data.
7. **admin():**
   Allows administrators to log in with credentials stored in a separate CSV file.
8. **admin_menu():**
   Displays the admin dashboard with options like showing users, viewing total assets, and deleting users.
9. **show_users():**
   Retrieves and displays a list of users, their account numbers, and dates of joining in a tabular format.
10. **total_assets():**
    Calculates and displays the total assets held by the bank from all user accounts.

11. **menu():**
    Displays the user menu with options like deposit, withdrawal, expense logging, and account details.
12. **deposit():**
    Allows users to deposit a specified amount into their account and updates the database accordingly.
13. **withdraw():**
    Enables users to withdraw funds, ensuring sufficient balance before deducting the amount.
14. **log_expenses():**
    Logs user expenses by recording category, amount, and transaction details in the database.
15. **bank_statement():**
    Retrieves and displays the transaction history for the logged-in user in a tabular format.
16. **account_details():**
    Displays detailed information about the logged-in user's account, including account number, balance, and date of joining.
17. **help():**
    Displays help information by reading content from an external help file.
18. **track_expenses():**
    Generates a bar graph showing the user's expenses over time, with categories and amounts visualized using matplotlib.

Source Code:

1. Importing Packages:

```
import mysql.connector as mycon
import random
import csv
from datetime import datetime, date
from tabulate import tabulate
from PIL import Image
import matplotlib.pyplot as plt
import time
from prettytable import PrettyTable
from collections import defaultdict
```

2. Database Connection & Cursor Creation:

```
# Database connection
con = mycon.connect(
    host="localhost",
    user="root",
    passwd="0000",
    database="finvault"
)

if con.is_connected():
    print("[LOGS] : Database Connected")

cur = con.cursor()
```

3. Welcome image display:

```
def display_image(image_path):
    image = Image.open(image_path)
    image.show()

image_path = r"C:\Users\dell\Desktop\fin-vault\fin-vault-img.png"
display_image(image_path)
```

4. Generate Account Number:

```
def generate_account(cursor):
    while True:
        account_number = random.randint(1000, 9999)

        cursor.execute(f"SELECT * FROM users WHERE account = {account_number}")
        if cursor.fetchone() is None:
            return account_number
```

5. Fin_Vault ASCII Art Generator:

```
def fin_vault():
        print(r"╔═══════════════════════════════════════════════════════════╗")
        print(r"│  ______  _                __      __                _  _  │")
        print(r"│ |  ____|(_)               \ \    / /               | || | │")
        print(r"│ | |__    _  _ __   ______  \ \  / /   __ _  _   _ | || |_ │")
        print(r"│ |  __|  | || '_ \ |______|  \ \/ /   / _` || | | || || __| │")
        print(r"│ | |     | || | | |            \  /   | (_| || |_| || || |_  │")
        print(r"│ |_|     |_||_| |_|             \/     \__,_| \__,_||_| \__| │")
        print(r"│                                                           │")
        print(r"╚═══════════════════════════════════════════════════════════╝")
```

6. Welcome Function:

```
def welcome():
    while True:
        fin_vault()
        time.sleep(0.18)
        print(r"╔═══════════════════════════════════════════════════════════╗")
        print(r"│           Welcome! How can we assist you today?           │")
        print(r"╚═══════════════════════════════════════════════════════════╝")
        time.sleep(0.18)
        print(r"╔═══════════════════════════════════════════════════════════╗")
        print(r"│                 1. Open an Account                        │")
        print(r"│                 2. Login to your Account                  │")
        print(r"│                 3. Admin Login                            │")
        print(r"│                 4. Exit the App                           │")
        print(r"╚═══════════════════════════════════════════════════════════╝")

        choice = input("Enter Your Choice: ")
        if choice == "1":
            signup()
            return
        elif choice == "2":
            login()
            return
        elif choice == "3":
            admin()
            return
```

```
        elif choice == "4":
            fin_vault()
            time.sleep(0.2)
            print(r"╔═══════════════════════════════════════════════════════╗")
            print(r"║              Thank You for using Fin - Vault          ╕")
            print(r"╚═══════════════════════════════════════════════════════╝")
            quit()
        else:
            print(r"╔═══════════════════════════════════════════════════════╗")
            print(r"║              Invalid Input, Please choose             ╕")
            print(r"║                 options from 1 - 4                    ╕")
            print(r"╚═══════════════════════════════════════════════════════╝")
```

7. Signup Function:

```
def signup():
    global username
    while True:
        print(r"╔═══════════════════════════════════════════════════════╗")
        print(r"║              Enter your desired username:             ╕")
        print(r"╚═══════════════════════════════════════════════════════╝")
        username = input("INPUT:")

        with open(r"C:\Users\dell\Desktop\fin-vault\data.csv", "r") as file:
            read = csv.reader(file)
            rows = list(read)
            items = [row[0] for row in rows]

        if username in items:
            time.sleep(0.18)
            print(r"╔═══════════════════════════════════════════════════════╗")
            print(r"║              Username already exists.                 ╕")
            print(r"╚═══════════════════════════════════════════════════════╝")
            continue
        if len(username) > 20:
            time.sleep(0.18)
            print(r"╔═══════════════════════════════════════════════════════╗")
            print(r"║              Username should be of max                ╕")
            print(r"║                 20 characters                         ╕")
            print(r"╚═══════════════════════════════════════════════════════╝")
            continue

        break

    while True:
        time.sleep(0.18)
        print(r"╔═══════════════════════════════════════════════════════╗")
        print(r"║              Enter a valid password:                  ╕")
        print(r"╚═══════════════════════════════════════════════════════╝")


        pwd = input("INPUT:")

        time.sleep(0.18)
        print(r"╔═══════════════════════════════════════════════════════╗")
        print(r"║              Confirm Password                         ╕")
        print(r"╚═══════════════════════════════════════════════════════╝")
```

```
        confirm_pwd = input("INPUT:")

        if pwd != confirm_pwd:
            time.sleep(0.18)
            print(r"╔═══════════════════════════════════════════════════════╗")
            print(r"║                   Passwords do not match              ╕")
            print(r"╚═══════════════════════════════════════════════════════╝")
            continue

        break
data = [username, pwd]
with open(r'C:\Users\dell\Desktop\fin-vault\data.csv', mode='a', newline='') as file:
    writer = csv.writer(file)
    writer.writerow(data)

unique_account = generate_account(cur)
doj = datetime.now()
cur.execute("INSERT INTO users (account, balance, doj, username) VALUES (%s, %s, %s, %s)",
            (unique_account, 0.00, doj, username))
cur.execute(f"CREATE TABLE `{username}` (id INT AUTO_INCREMENT PRIMARY KEY,"
"logs VARCHAR(20), balance DECIMAL(10, 2) DEFAULT 0.00, date DATE);")

con.commit()
print(r"╔═══════════════════════════════════════════════════════╗")
print(r"║                Account Created successfully!          ╕")
print(r"╚═══════════════════════════════════════════════════════╝")
time.sleep(1.0)
menu()
data = [username, pwd]
with open(r'C:\Users\dell\Desktop\fin-vault\data.csv', mode='a', newline='') as file:
    writer = csv.writer(file)
    writer.writerow(data)

unique_account = generate_account(cur)
doj = datetime.now()
cur.execute("INSERT INTO users (account, balance, doj, username) VALUES (%s, %s, %s, %s)",
            (unique_account, 0.00, doj, username))
cur.execute(f"CREATE TABLE `{username}` (id INT AUTO_INCREMENT PRIMARY KEY,"
"logs VARCHAR(20), balance DECIMAL(10, 2) DEFAULT 0.00, date DATE);")
    con.commit()
    print(r"╔═══════════════════════════════════════════════════════╗")
    print(r"║                Account Created successfully!          ╕")
    print(r"╚═══════════════════════════════════════════════════════╝")
    time.sleep(1.0)
    menu()
```

8. Login Function:

```
def login():
    global username
    while True:
        print(r"╔═══════════════════════════════════════════════════════════╗")
        print(r"│                    Enter your username:                   │")
        print(r"╚═══════════════════════════════════════════════════════════╝")
        input_username = input("INPUT: ")

        with open(r"C:\Users\dell\Desktop\fin-vault\data.csv", "r") as file:
            read = csv.reader(file)
            rows = list(read)
       user_exists = False
       for row in rows:
           if input_username == row[0]:
               user_exists = True
               attempt_count = 0
               while attempt_count < 3:
                   time.sleep(0.18)
                   print(r"╔═══════════════════════════════════════════════════════════╗")
                   print(r"│                   Enter your password:                    │")
                   print(r"╚═══════════════════════════════════════════════════════════╝")
                   password = input("INPUT: ")
                if password == row[1]:
                    time.sleep(0.18)
                    print(r"╔═══════════════════════════════════════════════════╗")
                    print(r"│                   Login Successful                │")
                    print(r"╚═══════════════════════════════════════════════════╝")
                    username = input_username
                    time.sleep(1.0)
                    menu()
                    return
                else:
                    attempt_count += 1
                    print(r"╔═══════════════════════════════════════════════════╗")
                    print(f"│      Invalid Password. {3 - attempt_count} attempts left.  │")
                    print(r"╚═══════════════════════════════════════════════════╝")
                    if attempt_count >= 3:
                        print(r"╔═══════════════════════════════════════════════╗")
                        print(r"│          Too many failed attempts. Please try again later.  │")
                        print(r"╚═══════════════════════════════════════════════╝")
                        time.sleep(1)
                        return

            break
```

9. Admin Function:

```
def admin():
    global adminusername
    while True:
        print(r"╔═══════════════════════════════════════════════════════════╗")
        print(r"│                    Enter your username:                   │")
        print(r"╚═══════════════════════════════════════════════════════════╝")
        input_username = input("INPUT: ")

        with open(r"C:\Users\dell\Desktop\fin-vault\admin.csv", "r") as file:
            read = csv.reader(file)
            rows = list(read)
```

```
        user_exists = False
        for row in rows:
            if input_username == row[0]:
                user_exists = True
                attempt_count = 0
                while attempt_count < 3:
                    time.sleep(0.18)
                    print(r"┌──────────────────────────────────────────────────────────┐")
                    print(r"│                    Enter your password:                  │")
                    print(r"└──────────────────────────────────────────────────────────┘")
                    password = input("INPUT: ")
                    if password == row[1]:
                        time.sleep(0.18)
                        print(r"┌──────────────────────────────────────────────────────────┐")
                        print(r"│                       Login Successful                   │")
                        print(r"└──────────────────────────────────────────────────────────┘")
                        username = input_username
                        time.sleep(1.0)
                        admin_menu()
                        return
                    else:
                        attempt_count += 1
                        print(r"┌──────────────────────────────────────────────────────────┐")
                        print(f"│      Invalid Password. {3 - attempt_count} attempts left.  │")
                        print(r"└──────────────────────────────────────────────────────────┘")
                        if attempt_count >= 3:
                            print(r"┌──────────────────────────────────────────────────────────┐")
                            print(r"│          Too many failed attempts. Please try again later. │")
                            print(r"└──────────────────────────────────────────────────────────┘")
                            time.sleep(1)
                            return
                    break
```

10. Admin menu:

```
def admin_menu():
    while True:
        fin_vault()
        time.sleep(0.18)
        print(r"┌──────────────────────────────────────────────────────────┐")
        print(r"│                                                          │")
        print(r"│                    1. Show Users                         │")
        print(r"│                                                          │")
        print(r"│                    2. Show Total Assets                  │")
        print(r"│                                                          │")
        print(r"│                    3. Delete Users                       │")
        print(r"│                                                          │")
        print(r"│                    4. Quit                               │")
        print(r"│                                                          │")
        print(r"└──────────────────────────────────────────────────────────┘")
        print("INPUT : ", end="")
        try:
            choice = int(input())
            if choice == 1:
                show_users()
            elif choice == 2:
                total_assets()
            elif choice == 3:
                fin_vault()
                time.sleep(0.2)
                print(r"┌──────────────────────────────────────────────────────────┐")
                print(r"│              Thank You for using Fin - Vault             │")
                print(r"└──────────────────────────────────────────────────────────┘")
                quit()
            else:
                print(r"┌──────────────────────────────────────────────────────────┐")
                print(r"│                   Invalid choice. Try again!             │")
                print(r"└──────────────────────────────────────────────────────────┘")
```

11. User Details:

```
def show_users():
    try:
        cur.execute(f"SELECT account,username,doj FROM users;")
        results = cur.fetchall()

        if len(results) == 0:
            print(r"╔═══════════════════════════════════════════════════════╗")
            print(r"║           [INFO] No Users found in your database.         ")
            print(r"╚═══════════════════════════════════════════════════════╝")
            return

        table = PrettyTable()
        table.field_names = ["Account Number", "Username", "Date of Joining"]

        for row in results:
            account, username, doj = row
            table.add_row([account, username, doj])

        print(r"╔═══════════════════════════════════════════════════════╗")
        print(r"║               Current User Date is below:                 ")
        print(r"╚═══════════════════════════════════════════════════════╝")
        print(table)
        time.sleep(5.0)

    except Exception as e:
        print(r"╔═══════════════════════════════════════════════════════╗")
        print(f"    [ERROR] An error occurred: {str(e)}.   ")
        print(r"╚═══════════════════════════════════════════════════════╝")
```

12. Total Assets:

```
def total_assets():
    try:
        cur.execute("SELECT SUM(balance) FROM users;")
        result = cur.fetchone()[0]

        table = PrettyTable()
        table.field_names = ["Total Assets Under the Bank"]

        if result is None:
            print(r"╔═══════════════════════════════════════════════════════╗")
            print(r"║           [INFO] No Assets found in your database.         ")
            print(r"╚═══════════════════════════════════════════════════════╝")
            return
        table.add_row([result])

        print(r"╔═══════════════════════════════════════════════════════╗")
        print(r"║               Total Assets Under The Bank:                 ")
        print(r"╚═══════════════════════════════════════════════════════╝")
        print(table)
        time.sleep(5.0)
    except Exception as e:
        print(r"╔═══════════════════════════════════════════════════════╗")
        print(f"    [ERROR] An error occurred: {str(e)}.   ")
        print(r"╚═══════════════════════════════════════════════════════╝")
```

13. Delete Users:

```python
def delete_users():
    try:
        cur.execute("SELECT account, username FROM users;")
        users = cur.fetchall()

        if not users:
            print("No users found in the database.")
            return

        table = PrettyTable()
        table.field_names = ["Account Number", "Username"]
        for user in users:
            table.add_row(user)

        print("Current Users:")
        print(table)


        account_to_delete = input("Enter the account number of the user to delete: ")

        cur.execute("SELECT username FROM users WHERE account = %s;", (account_to_delete,))
        user = cur.fetchone()

        if not user:
            print("Account not found. Please enter a valid account number.")
            return

        username = user[0]

        cur.execute(f"DROP TABLE IF EXISTS {username};")
        print(f"Table '{username}' has been deleted.")

        cur.execute("DELETE FROM users WHERE account = %s;", (account_to_delete,))
        con.commit()
        print(f"User '{username}' has been deleted from the 'users' table.")

        csv_file = r'C:\Users\dell\Desktop\fin-vault\data.csv'
        rows = []

        with open(csv_file, mode='r', newline='') as file:
            reader = csv.reader(file)
            for row in reader:
                if row and row[0] != username:
                    rows.append(row)

        with open(csv_file, mode='w', newline='') as file:
            writer = csv.writer(file)
            writer.writerows(rows)

        print(f"User '{username}' has been removed from the CSV file.")
        print("\nUser deletion process completed successfully.")

    except Exception as e:
        print(f"[ERROR] An error occurred: {str(e)}")
```

14. Main Menu:

```
def menu():
    current_date = date.today()

    while True:
        fin_vault()
        time.sleep(0.18)
        print(f"╔═══════════════════════════════════════════════════════════╗")
        print(f"                      Welcome {username}                    ")
        print(f"╚═══════════════════════════════════════════════════════════╝")
        time.sleep(0.18)
        print(r"╔═══════════════════════════════════════════════════════════╗")
        print(r"│                                                           │")
        print(r"│                    1. Deposit Balance                     │")
        print(r"│                                                           │")
        print(r"│                    2. Withdraw Balance                    │")
        print(r"│                                                           │")
        print(r"│                    3. Log Expenses                        │")
        print(r"│                                                           │")
        print(r"│                    4. Bank Statement                      │")
        print(r"│                                                           │")
        print(r"│                    5. Account Details                     │")
        print(r"│                                                           │")
        print(r"│                    6. About the App                       │")
        print(r"│                                                           │")
        print(r"│                    7. Track Expenses                      │")
        print(r"│                                                           │")
        print(r"│                    8. Quit                                │")
        print(r"│                                                           │")
        print(r"╚═══════════════════════════════════════════════════════════╝")
        print("INPUT : ", end="")
        user = username
        try:
            choice = int(input())
            if choice == 1:
                deposit()
            elif choice == 2:
                withdraw()
            elif choice == 3:
                log_expenses()
            elif choice == 4:
                bank_statement()
            elif choice == 5:
                account_details()
            elif choice == 6:
                help()
            elif choice == 7:
                track_expenses()
```

```
        elif choice == 8:
            fin_vault()
            time.sleep(0.2)
            print(r"╔═══════════════════════════════════════════════════════╗")
            print(r"║             Thank You for using Fin - Vault           ")
            print(r"╚═══════════════════════════════════════════════════════╝")
            quit()
        else:
            print(r"╔═══════════════════════════════════════════════════════╗")
            print(r"║             Invalid choice. Try again!                ")
            print(r"╚═══════════════════════════════════════════════════════╝")
    except ValueError:
        print(r"╔═══════════════════════════════════════════════════════╗")
        print(r"║             Please enter a valid number!              ")
        print(r"╚═══════════════════════════════════════════════════════╝")
```

15. Deposit Function:

```
def deposit():
    while True:
        print(r"╔═══════════════════════════════════════════════════════╗")
        print(r"║   Enter the amount you want to deposit: [exit to Exit]  ")
        print(r"╚═══════════════════════════════════════════════════════╝")
        amount = input("INPUT: AED ")

        if amount.lower() == 'exit':
            print(r"╔═══════════════════════════════════════════════════════╗")
            print(f"║             [EXIT] Cancelling Deposit.                ")
            print(r"╚═══════════════════════════════════════════════════════╝")
            time.sleep(1.0)
            break
    try:
        amount_float = int(amount)

        # Updating users table with new balance
        cur.execute("UPDATE users SET balance = balance + %s WHERE username = %s;", (amount_float, username))
        # Adding it into logs for that specifc user (will be accessible later)
        cur.execute(f"INSERT INTO {username} (logs, balance, date) VALUES (%s, %s, NOW());", (f"Deposited", amount))
        con.commit()

        print(r"╔═══════════════════════════════════════════════════════╗")
        print(f"║             Deposit of AED {amount_float} successful.      ")
        print(r"╚═══════════════════════════════════════════════════════╝")
        time.sleep(1.0)
        break
```

16. Withdraw Function:

```
def withdraw():
    while True:
        print(r"╔═══════════════════════════════════════════════════╗")
        print(r"│   Enter the amount you want to withdraw: [exit to Exit]   │")
        print(r"╚═══════════════════════════════════════════════════╝")
        amount = input("INPUT: AED ")

        if amount.lower() == 'exit':
            print(r"╔═══════════════════════════════════════════════╗")
            print(f"│            [EXIT] Cancelling Withdrawal.            │")
            print(r"╚═══════════════════════════════════════════════╝")
            time.sleep(1.0)
            break

        try:
            amount_float = int(amount)

            # Updating users table with new balance
            cur.execute("""UPDATE users SET balance = balance - %s WHERE username = %s AND balance >= %s;""",
                        (amount_float, username, amount_float))

            if cur.rowcount == 0:
                print(r"╔═══════════════════════════════════════════╗")
                print(f"│            [ERROR] Insufficient balance.            │")
                print(r"╚═══════════════════════════════════════════╝")
                continue

             # Adding it into logs with the name WITHDRAWN which can be viewed later.
             cur.execute(f"INSERT INTO {username} (logs, balance, date) VALUES (%s, %s, NOW());",
                         (f"Withdrawn", amount))
             con.commit()

             print(r"╔═══════════════════════════════════════════════════╗")
             print(f"│       Withdrawal of AED {amount_float} successful.       │")
             print(r"╚═══════════════════════════════════════════════════╝")
             time.sleep(1.0)
             break

         except ValueError:
             print(r"╔═══════════════════════════════════════════════════╗")
             print(r"│   [ERROR] Invalid input. Please enter a valid amount.   │")
             print(r"╚═══════════════════════════════════════════════════╝")

         except ValueError:
             print(r"╔═══════════════════════════════════════════════════╗")
             print(r"│   [ERROR] Invalid input. Please enter a valid amount.   │")
             print(r"╚═══════════════════════════════════════════════════╝")

         except Exception as e:
             print(r"╔═══════════════════════════════════════════════════╗")
             print(f"│   [ERROR] An error occurred: {str(e)}.   ")
             print(r"╚═══════════════════════════════════════════════════╝")
```

17. Log Expenses:

```
def log_expenses():
    while True:
        fin_vault()
        time.sleep(0.2)
        print(r"╔═══════════════════════════════════════════════════════════╗")
        print(r"║                      Log Your Expenses                    ")
        print(r"╚═══════════════════════════════════════════════════════════╝")
        time.sleep(0.2)

        # Get category input
        print(r"╔═══════════════════════════════════════════════════════════╗")
        print(r"║  Enter your expenditure category (ex: food, drinks, etc.)  ")
        print(r"║                    Type 'exit' to Exit                    ")
        print(r"╚═══════════════════════════════════════════════════════════╝")
        category = input("INPUT: ")

        if len(category) > 20:
            print(r"╔═══════════════════════════════════════════════════════╗")
            print(r"║      Category can be a maximum of 20 characters only  ")
            print(r"╚═══════════════════════════════════════════════════════╝")
            continue

        # Get amount input
        print(r"╔═══════════════════════════════════════════════╗")
        print(r"║            Enter your expenditure amount:     ")
        print(r"╚═══════════════════════════════════════════════╝")
        try:
            amount = float(input("INPUT: AED "))
            if amount <= 0:
                print(r"╔═══════════════════════════════════════════╗")
                print(r"║   [ERROR] Amount must be greater than zero.  ")
                print(r"╚═══════════════════════════════════════════╝")
                continue

        except ValueError:
            print(r"╔═══════════════════════════════════════════════╗")
            print(r"║  [ERROR] Invalid input. Please enter a valid amount.  ")
            print(r"╚═══════════════════════════════════════════════╝")
            continue

        try:
            cur.execute(f"INSERT INTO {username} (logs, balance, date) VALUES (%s, %s, NOW());",
                        (f"Spent On {category}", amount))
            con.commit()  # Save the changes

            print(r"╔═══════════════════════════════════════════════╗")
            print(f"║  Expense of AED {amount:.2f} for {category} logged successfully.  ║")
            print(r"╚═══════════════════════════════════════════════╝")
            time.sleep(2.0)
            break

        except Exception as e:
            print(r"╔═══════════════════════════════════════════════╗")
            print(f"║  [ERROR] Failed to log the expense: {str(e)}  ║")
            print(r"╚═══════════════════════════════════════════════╝")
```

18. Bank Statement:

```
def bank_statement():
    try:
        cur.execute(f"SELECT date, logs, balance FROM {username};")
        results = cur.fetchall()  # Fetch all results

        if len(results) == 0:
            print(r"╔═══════════════════════════════════════════════════════╗")
            print(r"║           [INFO] No transactions found in your account.    ╒")
            print(r"╚═══════════════════════════════════════════════════════╝")
            return

        # Create a PrettyTable to display the statement
        table = PrettyTable()
        table.field_names = ["Date of Transaction", "Transaction Details", "Amount Spent (AED)"]

        # Iterate through the results and add them to the table
        for row in results:
            date, logs, amount = row
            table.add_row([date, logs, f"{amount:.2f}"])

        # Print the statement in a neat format
        print(r"╔═══════════════════════════════════════════════════════╗")
        print(r"║                Your Bank Statement is Below:                ╒")
        print(r"╚═══════════════════════════════════════════════════════╝")
        print(table)
        time.sleep(5.0)

    except Exception as e:
        print(r"╔═══════════════════════════════════════════════════════╗")
        print(f"║        [ERROR] Failed to retrieve bank statement: {str(e)}  ╒")
        print(r"╚═══════════════════════════════════════════════════════╝")
```

19. Account Details:

```
def account_details():
    global username
    cur.execute(f"SELECT account, balance, doj, username FROM users WHERE username = '{username}';")
    results = cur.fetchall()

    table = PrettyTable()
    table.field_names = ["Account Number", "Username", "Amount (AED)", "Date of Joining"]

    for row in results:
        account, balance, doj, username = row
        table.add_row([account, username, balance, doj])

    print(r"╔═══════════════════════════════════════════════════════╗")
    print(r"║              Your Account Details Are Below:              ╒")
    print(r"╚═══════════════════════════════════════════════════════╝")
    print(table)
    time.sleep(5.0)
```

20. Help Information:

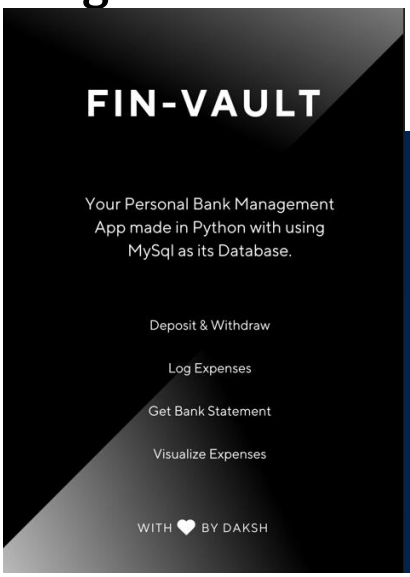
```
def help():
    try:
        with open('help_file.txt', 'r') as file:
            contents = file.read()

        print(r"╔═══════════════════════════════════════════════════╗")
        print(r"║                   Help Information                ╕")
        print(r"╚═══════════════════════════════════════════════════╝")
        print(contents)
        print(r"╔═══════════════════════════════════════════════════╗")
        print(r"║                End of Help Information            ╕")
        print(r"╚═══════════════════════════════════════════════════╝")
        time.sleep(10.0)

    except FileNotFoundError:
        print("Error: Help file not found.")
```

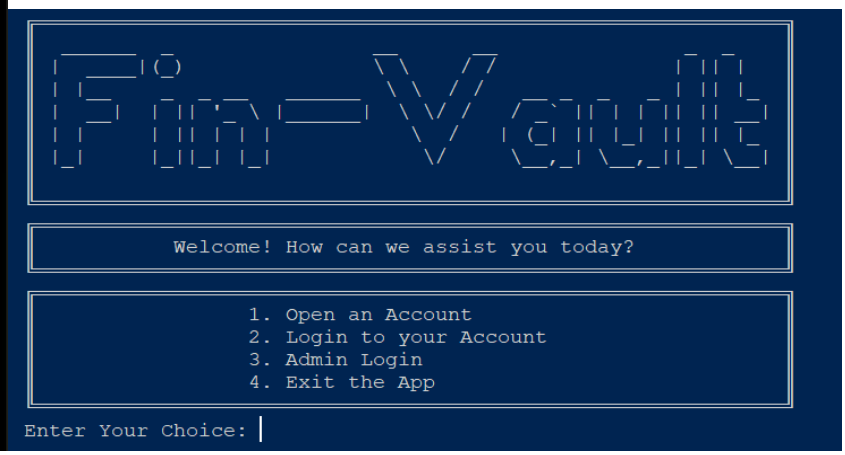
21. Tracking Expenses:

```
def track_expenses():
    try:
        cur.execute(f"SELECT date, logs, balance FROM {username} WHERE logs LIKE '%spent%';")
        results = cur.fetchall()

        if len(results) == 0:
            print(r"╔═══════════════════════════════════════════════════╗")
            print(r"║          [INFO] No expense transactions found.    ╕")
            print(r"╚═══════════════════════════════════════════════════╝")
            return

        dates = []
        amounts = []

        for row in results:
            date_obj, logs, amount = row
            dates.append(f"{logs}")
            amounts.append(amount)

        print(r"╔═══════════════════════════════════════════════════╗")
        print(r"║          Expense Tracking Graph Generated Below.  ╕")
        print(r"╚═══════════════════════════════════════════════════╝")
        time.sleep(1.0)

        plt.figure(figsize=(12, 6))
        plt.bar(dates, amounts, color='lightcoral', label='Amount Spent (AED)')
        plt.xticks(rotation=90, ha='center', fontsize=8)
        plt.xlabel('Transaction (Date & Details)')
        plt.ylabel('Amount Spent (AED)')
        plt.title('Expenses for Each Transaction Over Time')
        plt.grid(True, axis='y')
        plt.tight_layout()
        plt.legend()
        plt.show()

    except Exception as e:
        print(r"╔═══════════════════════════════════════════════════╗")
        print(f"║     [ERROR] Failed to retrieve expenses: {str(e)}  ╕")
        print(r"╚═══════════════════════════════════════════════════╝")
```

Output Screen:

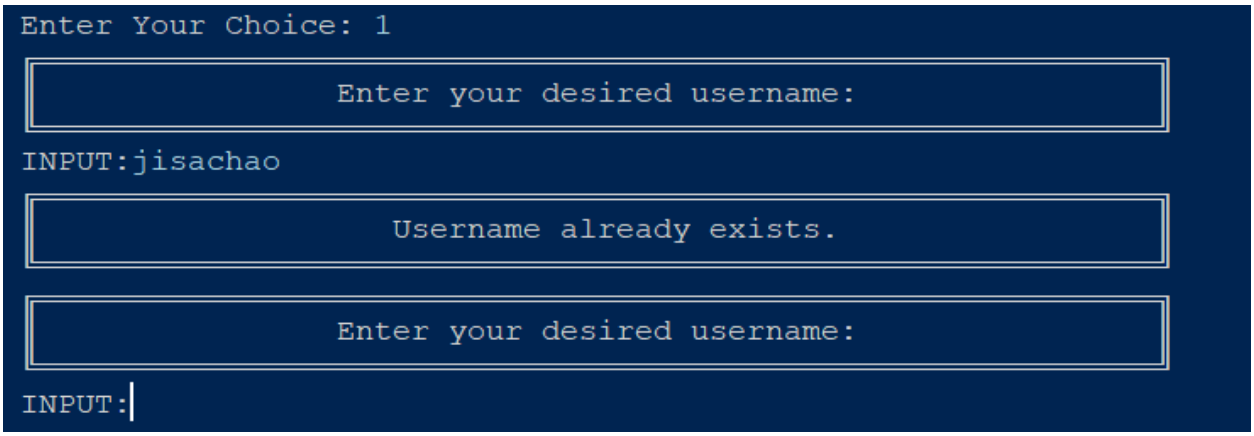
Program Run:





Opening An Account:

If Username Exists:



If Password doesn't match:

```
Enter your desired username:
INPUT:haha
Enter a valid password:
INPUT:1234
Confirm Password
INPUT:sjjs
Passwords do not match
```

If Passwords match:

```
Enter a valid password:
INPUT:1234
Confirm Password
INPUT:1234
Account Created successfully!
```

Data.csv file:

haha	1234

SQL:

```
mysql> show tables;
+--------------------+
| Tables_in_finvault |
+--------------------+
| clxvdyval          |
| haha               |
| jisa               |
| jisachao           |
| rehan              |
| users              |
+--------------------+
6 rows in set (0.01 sec)
```

```
mysql> select * from users;
+---------+---------+------------+-----------+
| account | balance | doj        | username  |
+---------+---------+------------+-----------+
| [illegible] | [illegible] | 2024-09-23 | resh      |
| 3962    | 280.00  | 2024-12-16 | haha      |
| [illegible] | 400.00  | 2024-09-17 | jisachao  |
| 8103    | 1000.00 | 2024-09-18 | clxvdyval |
| 9774    | 0.00    | 2024-09-18 | jisa      |
+---------+---------+------------+-----------+
5 rows in set (0.00 sec)
```

Logging into account:

If Password wrong (3 tries):

```
Enter Your Choice: 2

                    Enter your username:

INPUT: haha

                    Enter your password:

INPUT: 123

      Invalid Password. 2 attempts left.

                    Enter your password:

INPUT: hahaa

      Invalid Password. 1 attempts left.

                    Enter your password:

INPUT: haha

      Invalid Password. 0 attempts left.

          Too many failed attempts. Please try again later.
```

If Username doesn't exist:

```
                    Enter your username:

INPUT: lskdjfgn

                    Username not found!

            Do you want to create a new account
                  (y) - yes . (n) - no
```

```
INPUT: y

Enter your desired username:

INPUT:jisa3392

Enter a valid password:

INPUT:
```

(moves to signup function)

Main Menu:

```
Welcome haha

1. Deposit Balance
2. Withdraw Balance
3. Log Expenses
4. Bank Statement
5. Account Details
6. About the App
7. Track Expenses
8. Quit
```

Deposit Balance:

If Exiting:

```
Enter the amount you want to deposit: [exit to Exit]

INPUT: AED exit

[EXIT] Cancelling Deposit.
```

If Successful:

```
INPUT : 1
╔═══════════════════════════════════════════════════════════╗
   Enter the amount you want to deposit: [exit to Exit]
╚═══════════════════════════════════════════════════════════╝
INPUT: AED 20
╔═══════════════════════════════════════════════════════════╗
                 Deposit of AED 20 successful.
╚═══════════════════════════════════════════════════════════╝
```

```
mysql> select * from users;
+---------+---------+------------+----------+
| account | balance | doj        | username |
+---------+---------+------------+----------+
|    8162 |   20.00 | 2024-12-18 | jisachao |
+---------+---------+------------+----------+
1 row in set (0.00 sec)
```

Withdraw Balance:

If Balance Insufficient:

```
╔═══════════════════════════════════════════════════════════╗
   Enter the amount you want to withdraw: [exit to Exit]
╚═══════════════════════════════════════════════════════════╝
INPUT: AED 2
╔═══════════════════════════════════════════════════════════╗
              [ERROR] Insufficient balance.
╚═══════════════════════════════════════════════════════════╝
╔═══════════════════════════════════════════════════════════╗
   Enter the amount you want to withdraw: [exit to Exit]
╚═══════════════════════════════════════════════════════════╝
```

If Successful:

```
╔═══════════════════════════════════════════════════════════╗
   Enter the amount you want to withdraw: [exit to Exit]
╚═══════════════════════════════════════════════════════════╝
INPUT: AED 20
╔═══════════════════════════════════════════════════════════╗
       Withdrawal of AED 20 successful.
╚═══════════════════════════════════════════════════════════╝
```

```
mysql> select * from users;
+---------+---------+------------+----------+
| account | balance | doj        | username |
+---------+---------+------------+----------+
|    8162 |    0.00 | 2024-12-18 | jisachao |
+---------+---------+------------+----------+
1 row in set (0.00 sec)
```

Logging Expenses:

```
INPUT : 3

 _____  _                 __      __                _  _
|  ____|(_)               \ \    / /               | ||_|
| |__    _  _ __   _____  \ \  / /  __ _  _   _  | || |_
|  __|  | || '_ \ |_____|  \ \/ /  / _` || | | || || __|
| |     | || | | |          \  /  | (_| || |_| || || |_
|_|     |_||_| |_|           \/    \__,_| \__,_||_| \__|

                    Log Your Expenses

  Enter your expenditure category (ex: food, drinks, etc.)
                  Type 'exit' to Exit

INPUT: food

                Enter your expenditure amount:

INPUT: AED 20

    Expense of AED 20.00 for food logged successfully.
```

```
mysql> select * from jisachao;
+----+---------------+---------+------------+
| id | logs          | balance | date       |
+----+---------------+---------+------------+
|  1 | Deposited     |   20.00 | 2024-12-18 |
|  2 | Withdrawn     |   20.00 | 2024-12-18 |
|  3 | Spent On food |   20.00 | 2024-12-18 |
+----+---------------+---------+------------+
3 rows in set (0.00 sec)
```

Bank Statement:

```
Your Bank Statement is Below:

+---------------------+---------------------+--------------------+
| Date of Transaction | Transaction Details | Amount Spent (AED) |
+---------------------+---------------------+--------------------+
|      2024-12-16     |      Deposited      |       20.00        |
|      2024-12-16     |      Deposited      |        0.00        |
|      2024-12-16     |      Deposited      |       -20.00       |
|      2024-12-16     |      Deposited      |       300.00       |
|      2024-12-16     |      Withdrawn      |       20.00        |
|      2024-12-16     |    Spent On food    |       20.00        |
|      2024-12-16     |   Spent On drinks   |       50.00        |
+---------------------+---------------------+--------------------+
```

Account Details:

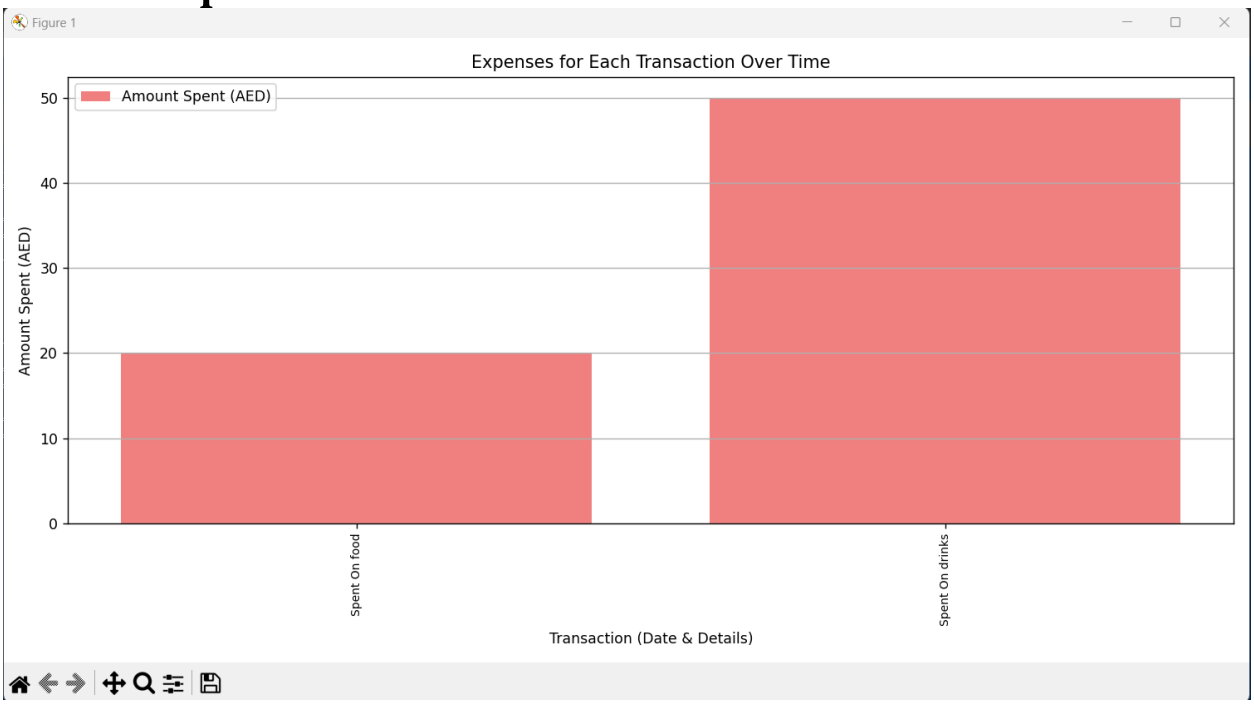
```
Your Account Details Are Below:

+----------------+----------+--------------+-----------------+
| Account Number | Username | Amount (AED) | Date of Joining |
+----------------+----------+--------------+-----------------+
|      3962      |   haha   |    280.00    |    2024-12-16   |
+----------------+----------+--------------+-----------------+
```

About the App:

```
Fin-Vault is a program which is meant to
act as your Personal Bank Management App.
It gives you features like depositing and
withdrawing balance (with no assets backing it)
You can also log your expenses and view your
bank statement.
```

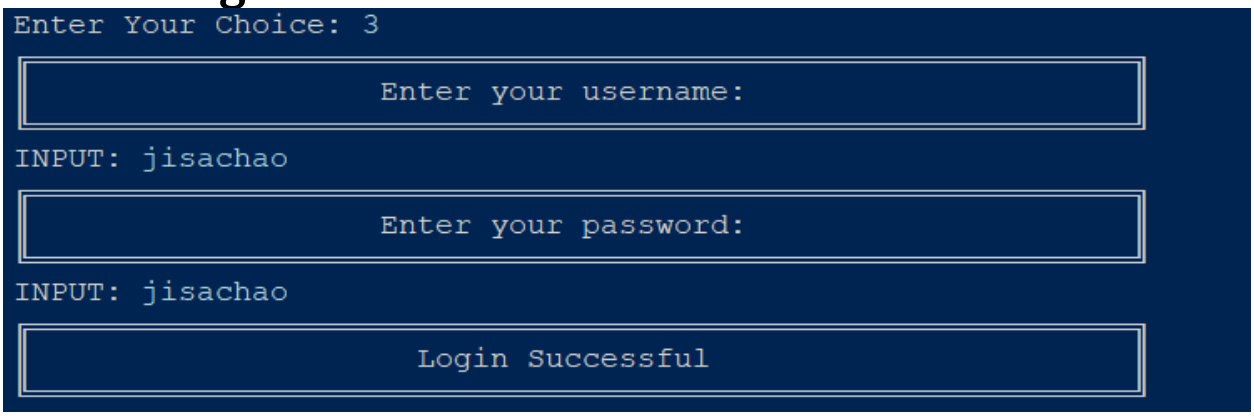
```
Help Information

Fin-Vault is a program which is meant to
act as your Personal Bank Management App.
It gives you features like depositing and
withdrawing balance (with no assets backing it)
You can also log your expenses and view your
bank statement.

End of Help Information
```

Track Expenses:



Quit:

Admin Login:



Admin Menu:

```
1. Show Users

2. Show Total Assets

3. Delete Users

4. Quit
```

Show Users:

```
Current User Date is below:

+----------------+-----------+-----------------+
| Account Number |  Username | Date of Joining |
+----------------+-----------+-----------------+
|      3638      |    resh   |    2024-09-23   |
|      3962      |    haha   |    2024-12-16   |
|      6849      |  jisachao |    2024-09-17   |
|      8103      | clxvdyval |    2024-09-18   |
|      9774      |    jisa   |    2024-09-18   |
+----------------+-----------+-----------------+
```

Total Assets:

If No Assets:

```
[INFO] No Assets found in your database.
```

If Assets Available:

```
INPUT : 2
+-----------------------------+
| Total Assets Under the Bank |
+-----------------------------+
|           2000.00           |
+-----------------------------+
```

Delete Users:

```
INPUT : 3
Current Users:
+----------------+----------+
| Account Number | Username |
+----------------+----------+
|      1234      | jisachao |
+----------------+----------+
Enter the account number of the user to delete: 1234
 Table 'jisachao' has been deleted.
 User 'jisachao' has been deleted from the 'users' table.
 User 'jisachao' has been removed from the CSV file.

 User deletion process completed successfully.
```

Before & After [Users Table Data]:

```
mysql> select * from users;
+---------+---------+------------+----------+
| account | balance | doj        | username |
+---------+---------+------------+----------+
|    1234 | 2000.00 | 2024-12-17 | jisachao |
+---------+---------+------------+----------+
1 row in set (0.00 sec)

1 row in set (0.00 sec)

mysql> select * from users;
Empty set (0.00 sec)
```

Before & After [Tables in fin_vault]

```
mysql> show tables;
+--------------------+
| Tables_in_finvault |
+--------------------+
| jisachao           |
| users              |
+--------------------+
2 rows in set (0.00 sec)
```

```
mysql> show tables;
+--------------------+
| Tables_in_finvault |
+--------------------+
| users              |
+--------------------+
1 row in set (0.00 sec)
```

Before & After [CSV]

1		
2	jisachao	1234
3		

1		
2		

Limitations

1. **Lack of Password Hashing**:
   The passwords entered by users are stored in plain text without any encryption or hashing, making them vulnerable to potential security breaches. A more secure approach would involve hashing the passwords before storing them in the database.
2. **No Multi-factor Authentication**:
   The system does not implement multi-factor authentication (MFA) for user or admin logins. MFA would add an extra layer of security by requiring a second form of authentication, such as a one-time code sent to the user's email or phone.
3. **Lack of User Session Management**:
   The system does not include proper session management for users after they log in. Once logged in, there is no mechanism to track active sessions or automatically log out users after a period of inactivity, which could potentially lead to unauthorized access if the user forgets to log out.

REQUIREMENTS

- HARDWARE REQUIRED

  - **Printer**: to print the required documents of the project
  - **Processor** : 12th Gen Intel(R) Core(TM) i7-12700H 2.30 GHz
  - **RAM** : 32 MB
  - **Hard Disk** : 20 Gb

- SOFTWARE REQUIRED

  - **Operating system** : Windows 11 Pro .
  - **Programming Language**: Thonny (Python) (Front End).
  - **Database Management Systems**: MySql (Back End).
  - **Application Software**: Ms Word, for presentation of output.
  - **Spreadsheet Software**: Ms Excel (Spreadsheet), for CSV file.

BIBLIOGRAPHY

http://www.google.com

http://en.wikipedia.org

Computer Science with Python: PREETI ARORA

Computer Science with Python: SUMITA ARORA